

3.12-3.18 Weekly Record

Data Chapter

1. Introduction
2. Dataset Collection and Sources
3. **Data Cleaning and Preprocessing**
4. Feature Representation

Review Integration - Sampling

```
yelp_academic_dataset_review.json: {'size_MB': 5094.4, 'num_records': 699028}  
yelp_academic_dataset_tip.json: {'size_MB': 172.24, 'num_records': 908915}
```

Process-heavy for 6616 businesses. Sample a subset

Proportionally sampling 200 businesses based on the 7 category clusters ensures a balanced representation of different business types.

Cluster	Original Counts	Proportion	Sampled
1 - Latin American	267	4.03%	8
2 - Asian	717	10.84%	22
3 - Global	298	4.51%	9
4 - American	2542	38.42%	77
5 - Bars	491	7.42%	15
6 - Street Food	956	14.46%	29
7 - Sweet Treats & Casual Eats	1345	20.32%	40
Total	6616	100%	200

▼ stats:

```
"review_statistics": {
  "Total Reviews": 22798,
  "Stars Distribution": {
    "5": 10083,
    "4": 6208,
    "3": 2620,
    "1": 2075,
    "2": 1812
  },
  "Review Count Per Business": {
    "mean": 113.99,
    "median": 59.5,
    "min": 10,
    "max": 1543,
    "std_dev": 177.64796621408306
  },
  "Text Length": {
    "mean": 558.3538029651723,
    "median": 409.0,
    "min": 3,
    "max": 5000,
    "std_dev": 496.7103866832614
  },
  "Review Date": {
    "earliest": "2005-06-27 14:21:34",
    "latest": "2022-01-19 15:44:43"
  }
},
"tip_statistics": {
  "Total Tips": 2936,
  "Tip Count Per Business": {
    "mean": 15.291666666666666,
    "median": 8.0,
    "min": 1,
    "max": 208,
```

```

    "std_dev": 23.56193817485216
  },
  "Text Length": {
    "mean": 58.03405994550409,
    "median": 47.0,
    "min": 1,
    "max": 496,
    "std_dev": 50.61321612998522
  },
  "Tip Date": {
    "earliest": "2009-04-17 22:00:26",
    "latest": "2022-01-16 22:50:49"
  },
  "Compliment Count": {
    "mean": 0.012942779291553134,
    "median": 0.0,
    "min": 0,
    "max": 2,
    "std_dev": 0.13247502980542872,
    "distribution": {
      "0": 2905,
      "1": 24,
      "2": 7
    }
  }
}

```

Attach at most **10 most recent reviews** and **10 most recent tips** per business to a new field **"text"** in the **200 sampled businesses**

New entry:

```
{
  "business_id": "YuVILLafdFUt9o2GexaxGg",
  "name": "El Limon Havertown ",
  "stars": 4.0,
  "review_count": 10,
  "category_cluster": 1,
  "description": "El Limon Havertown : categorised under Mexican",
  "RestaurantsTakeOut": 1,
  "BusinessParking": 0,
  "BikeParking": 0,
  "RestaurantsDelivery": 1,
  "RestaurantsReservations": 0,
  "OutdoorSeating": 1,
  "normalized_price": 0.3333333333,
  "text": [
    "Favorite casual Mexican in the West Philly suburbs. Love the Chips, salsa and",
    "This is my new go to Mexican restaurant. I've never had anything bad here and",
    "Tried el limon in havertown. Had a bad experience at the Bryn Mawr one previously",
    "We may be from West Chester, but for our anniversary my bride wanted to go",
    "Love the tinga enchiladas with flour tortilla and mole! My only wish is El Limon",
    "We love this place! \n\nGood food. VERY fast service. Staff is friendly and helpful",
    "Great food! So happy they have one now in havertown...Grilled chicken burrito",
    "Due to COVID, we haven't been able to dine in yet, but have gotten take out :)",
    "Tonight we take out again from El limon. We ordered Tamale, guacamole, chicken",
    "If I could give it zero stars I would. Worst experience with take out yet. Waited",
    "This place is amazing! Best El Limon in the area. Food never disappoints.",
    "Pretty good but not the best presentation",
    "Now open."
  ],
  "qual_pop_wt_score": 3.935,
  "nm_wt_score": 0.4999999999999998
}
```

A simple attempt at calculating

similarity based on the **user query**, without weighting or user profiles.
(output_recommendations\query_results_without_weights.json)

```
{
  "user_query": "Need a cheap burger spot with fast takeout.",
  "top_recommendations": [
    {
      "business_id": "C-JAsI-tgmf_eRmMXTp9Ew",
      "name": "Shake Shack",
      "description": "Shake Shack: a moderately priced restaurant categorised under American (",
      "cosine_similarity": 0.5051982998847961,
      "RestaurantsTakeOut": 1,
      "BusinessParking": 0,
      "BikeParking": 0,
      "RestaurantsDelivery": 0,
      "RestaurantsReservations": 0
    },
    {
      "business_id": "yiaCtwtl9xrsQtVwB96krQ",
      "name": "Country Place",
      "description": "Country Place: a budget-friendly restaurant with casual ambience categor",
      "cosine_similarity": 0.48586025834083557,
      "RestaurantsTakeOut": 1,
      "BusinessParking": 1,
      "BikeParking": 0,
      "RestaurantsDelivery": 0,
      "RestaurantsReservations": 0
    }
  ]
}
```

Utilize Each Field of Item

1. Structured Data (7):

```
"category_cluster": 1,
"RestaurantsTakeOut": 1,
"BusinessParking": 0,
"BikeParking": 0,
"RestaurantsDelivery": 1,
"RestaurantsReservations": 0,
"normalized_price": 0.3333333333,
```

- **category_cluster** → Encodes general cuisine type.

- **Binary attributes (RestaurantsTakeOut, BusinessParking, etc.)** → Used as features in a structured vector.
- **normalized_price** → Helps match users with price preferences.

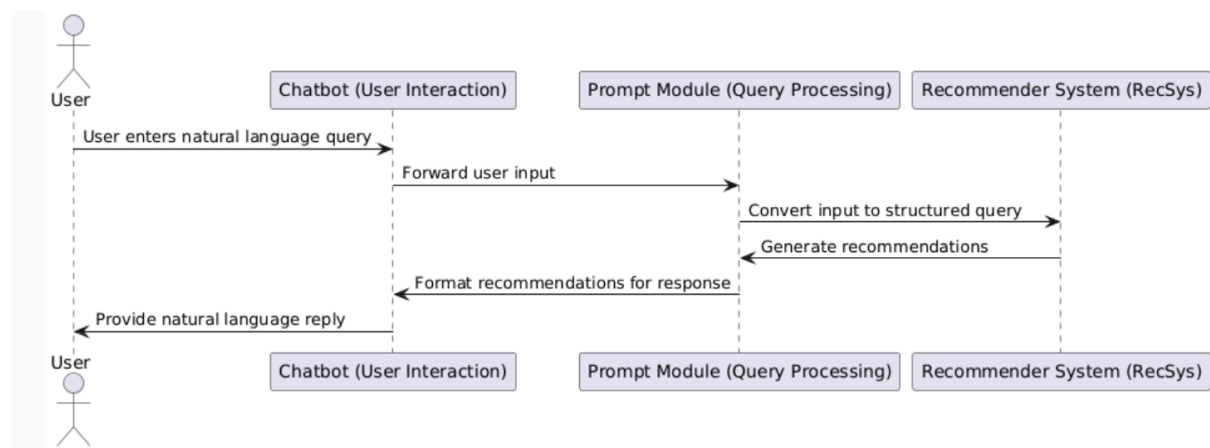
Usage in Model:

Construct **item embeddings** (vectors) from structured data.

Q: For User Profile: Fill a structured form like checkboxes (e.g., "Takeout: Yes, Parking: No")? Autofill by parsing user input?

Split into three parts: the **Chatbot** handles user interaction (queries, preferences, casual conversation), the **Prompt Module (Query Processing Layer)** reformats user input into structured queries, and the **Recommender System** generates recommendations. The output is then converted back into natural language by Chatbot.

Just focus on **Prompt Module-RecSys** part now



2. Unstructured Data:

"description": "El Limon Havertown : categorised under Mexican"

"text": [

"Favorite casual Mexican in the West Philly suburbs. Love the Chips, salsa and
 "This is my new go to Mexican restaurant. I've never had anything bad here and
 "Tried el limon in havertown. Had a bad experience at the Bryn Mawr one place
 "We may be from West Chester, but for our anniversary my bride wanted to go
 "Love the tinga enchiladas with flour tortilla and mole! My only wish is El Limon
 "We love this place! \n\nGood food. VERY fast service. Staff is friendly and h

```

"Great food! So happy they have one now in havertown...Grilled chicken burri
"Due to COVID, we haven't been able to dine in yet, but have gotten take out :
"Tonight we take out again from El limon. We ordered Tamale, guacamole, ch
"If I could give it zero stars I would. Worst experience with take out yet. Waite
"This place is amazing! Best El Limon in the area. Food never disappoints.",
"Pretty good but not the best presentation",
"Now open."
]

```

// keep the important text only

- **description** → Gives a short summary of the restaurant.
- **text (reviews & tips)** → Encodes **user sentiment and experience**.

Usage in Model:

- Convert text into **word embeddings (384 dimensions)**.
- Use **text-based similarity** for recommendations.

3. Weight:

- Use `nm_wt_score` to **scale similarity scores** when ranking items.

▼ qual_pop_wt_score

Previous:

`qual_pop_wt_score =`

$$\frac{\text{stars} \times \text{review_count} + w \times \text{global_avg_rating}}{\text{review_count} + w}$$

Global Average Rating: 3.8511

Smoothing Factor: 50.45

`median(df["review_count"])`

▼ Impact of Different `w` Values

- **Larger w (e.g., 1000+)** → Relies more on the global average rating s , reducing the influence of `review_count`.
- **Smaller w (e.g., 10)** → `review_count` has a greater impact, allowing popular businesses to dominate rankings.

1. Solve the skewed issue

Current the **Global Average Rating** is skewed due to filtering (`rating ≥ 3` , `review_count ≥ 10`).

Recalculate it using the dataset that are **before applying** the filtering conditions (`rating ≥ 3` , `review_count ≥ 10`).

Global Average Rating: 3.5608

Optimized Smoothing Factor: 32

`median(df["review_count"])`

2. Normalization (Min-Max Normalization)

- **Ensures consistent value range** → Makes scores comparable across different datasets.
- **Reduces the impact of outliers** → Prevents extreme values from dominating the analysis.
- **Improves numerical stability** → Helps algorithms process data more efficiently.
- **Optimizes for machine learning models** → Many models work better with normalized data.

Result:

rating_category	review_category	
high_rating	high_review	0.730520
	low_review	0.648017
low_rating	high_review	0.474871
	low_review	0.495928

Counterintuitive: low-rated businesses with fewer reviews score higher than those with more reviews.

3. Take the log of `review_count`

✓ Reduce the Impact of `review_count`. Avoid **high-review businesses** disproportionately affecting the ranking.

`qual_pop_wt_score =`

$$\frac{\text{stars} \times \log_review_count + w \times \text{global_avg_rating}}{\log_review_count + w}$$

Result:

rating_category	review_category	
high_rating	high_review	0.731565
	low_review	0.685277
low_rating	high_review	0.464406
	low_review	0.448386

The ranking aligns with expectations, initially effective

(Not yet further validate its effectiveness. *Score Distribution?*)

TODO: Sentiment Analysis

Next

Write Data Chapter

Run Small-Scale Models First as soon

Request Access to School GPU Server

Learn sentiment analysis